

Síntesis de teoría de la asignatura

Git

Git es un sistema distribuido que permite gestionar versiones, trabajar en equipo y mantener la historia completa de un proyecto. Para ello, utiliza cinco comandos fundamentales:

- **git init (Inicialización):** Activa Git en una carpeta local creando un directorio oculto `.git`. Es el punto de partida que permite rastrear cambios incluso sin conexión.
- **git add (Preparación):** Mueve los archivos desde el área de trabajo al área de preparación (*Staging Area*). Permite tomar una decisión consciente sobre qué cambios exactos se van a incluir, evitando subir archivos innecesarios.
- **git commit (Confirmación):** Crea un punto de control en el historial del desarrollo. Guarda una fotografía exacta del proyecto (qué, quién, cuándo y por qué) y siempre debe acompañarse de un mensaje claro y descriptivo.
- **git push (Publicación):** Sube los *commits* locales al repositorio remoto (como GitHub o GitLab) para sincronizar y compartir el trabajo individual con el resto del equipo.
- **git pull (Sincronización):** Combina la descarga de actualizaciones (*fetch*) con su fusión (*merge*). Actualiza tu repositorio local con las modificaciones más recientes del equipo para evitar sobrescribir el trabajo de otros.

Ramas (Branches)

Representan líneas independientes de desarrollo dentro de un proyecto; técnicamente, cada rama es un puntero que apunta a un conjunto de commits.

- **Naturaleza Distribuida:** Cada usuario tiene una copia completa del repositorio incluyendo su historial sus ramas, lo que permite trabajar sin conexión y experimentar sin riesgo.
- **Utilidad Principal:** Permiten aislar desarrollos, probar nuevas ideas o corregir errores sin afectar a la versión principal y estable del proyecto (generalmente llamada *main* o *master*).

Fusiones (Merges) y Conflictos

El comando **Merge** se utiliza para combinar ramas, fusionando los cambios de una rama en otra mientras se mantiene intacta la historia completa de ambas.

- **Fusión Automática vs. Conflicto:** Si los cambios se realizan en archivos diferentes, Git los combina automáticamente. Sin embargo, si dos ramas modifican exactamente las mismas líneas del mismo archivo, se produce un conflicto.
- **Resolución de Conflictos:**
 1. Git pausa la fusión y marca las diferencias en el archivo usando delimitadores visuales (<<<<<< HEAD, =====, >>>>>>).
 2. El desarrollador debe editar el archivo manualmente para decidir qué versión conservar (o cómo combinarlas) y borrar los delimitadores.
 3. Una vez resuelto, se debe ejecutar `git add` para marcar el archivo como solucionado y `git commit` para finalizar la fusión de forma definitiva.

Estrategias de Branching (Flujos de Trabajo)

Las estrategias definen cómo se organizan las ramas y cómo fluye el trabajo en el equipo:

- **Git Flow:**
 - Utiliza múltiples ramas (main, develop, feature, release, hotfix).
 - Ofrece un control exhaustivo, ideal para software empresarial con ciclos de liberación muy largos (ej. Siemens).
 - Es más complejo y menos ágil para hacer despliegues frecuentes.
- **GitHub Flow:**
 - Flujo sencillo basado en una rama main y ramas de características (feature/*) que se integran mediante *Pull Requests*.
 - Ideal para integración continua (CI/CD), despliegues rápidos y aplicaciones web (ej. Netflix).
- **Trunk Based Development:**

- Todos los desarrolladores trabajan directamente sobre main realizando commits diarios muy frecuentes.
- Requiere un altísimo nivel de testing automatizado y madurez DevOps (ej. Google).
- **Hotfix:**
 - Son ramas de emergencia y temporales creadas exclusivamente para corregir errores críticos directamente en producción de forma muy rápida (ej. procesos críticos en GitLab).

Comandos Clave en el Manejo de Ramas

- **Crear y cambiar a una nueva rama:** `git switch -c nombre-de-la-rama.`
- **Fusionar una rama (desde main):** `git merge nombre-de-la-rama.`
- **Eliminar una rama localmente:** `git branch -d nombre-de-la-rama.`
- **Ver el historial gráfico de ramas:** `git log --oneline --graph --decorate --all.`

Evaluación de Competencias Profesionales

Al dominar estos conceptos, un desarrollador debe cumplir con cuatro pilares:

1. **Uso técnico:** Aplicar comandos de ramas y merges sin cometer errores.
 2. **Colaboración:** Participar activamente en el flujo del equipo coordinando ramas y revisiones mediante *Pull Requests*.
 3. **Resolución:** Identificar, solucionar y documentar correctamente los conflictos de código.
 4. **Documentación:** Escribir mensajes de commit y de PR que sean concisos, descriptivos y normalizados.
-

Scrum: Roles, Artefactos y Eventos

Scrum

Es un marco de trabajo ágil diseñado para organizar proyectos complejos de una manera flexible, iterativa y colaborativa en el que el trabajo en equipo se organiza para desarrollar un producto a través de entregas pequeñas y frecuentes en ciclos cortos llamados **Sprints**, donde hay una planificación, ejecución, revisión y mejora continua.

Los Tres Pilares (Principios)

- **Transparencia:** Todos los miembros del equipo conocen el estado del proyecto y las tareas en curso.
- **Inspección:** El trabajo se revisa con frecuencia.
- **Adaptación:** Si algo falla o no funciona bien, el proceso se mejora y se cambia.

Roles

El trabajo se reparte en tres roles con funciones específicas:

- **Product Owner (PO):** Representa al cliente. Su responsabilidad es definir qué se va a hacer y priorizar la lista de requisitos (Product Backlog).
- **Scrum Master (SM):** Es el guía del equipo. Se encarga de asegurar que se sigan las prácticas de Scrum y de eliminar cualquier obstáculo que bloquee a los desarrolladores.
- **Development Team (DT):** Es el equipo que construye el producto. Se encargan de desarrollar, probar y entregar el producto funcional.

Artefactos

Representan el trabajo en el proyecto:

- **Product Backlog:** Es la lista priorizada de los requisitos e historias de usuario del producto.
- **Sprint Backlog:** Es el subconjunto de tareas del Product Backlog que se eligen para completarse en el Sprint actual.

- **Incremento:** Es el resultado real y funcional que se obtiene al terminar el Sprint.

Eventos

- **Sprint Planning:** Ocurre al inicio del Sprint y sirve para definir los objetivos.
- **Daily Scrum:** Es una reunión diaria y breve para revisar qué se hizo el día anterior, qué se hará hoy y si existen problemas.
- **Sprint Review:** Se realiza al final del Sprint para mostrar el incremento de producto terminado al Product Owner.
- **Sprint Retrospective:** Ocurre justo después de la Review; el equipo analiza cómo fue su forma de trabajar y qué cosas pueden mejorar para el siguiente Sprint.

Ventajas

- Permite realizar entregas de producto frecuentes y funcionales.
 - Aumenta la comunicación y la motivación dentro del equipo.
 - Ofrece flexibilidad para adaptarse a cambios o peticiones del cliente.
 - Fomenta el aprendizaje constante y la mejora continua del equipo.
 - Resulta en una mayor calidad del producto final.
-

Kanban, XP y Lean Development

Además de Scrum, existen otros enfoques ágiles que las empresas suelen combinar para mejorar sus procesos de desarrollo. Aunque todos sirven para organizar o mejorar el trabajo, cada método lo hace desde una perspectiva:

KANBAN – VISUALIZACIÓN Y FLUJO CONTINUO

Consiste en organizar el trabajo mediante un un tablero visual y tarjetas.

Elementos clave:

- **Tablero:** Lugar (físico o digital) donde se visualizan las tareas y su estado.
 - **Backlog:** historias de usuario.
 - **To Do:** HU que trabajaremos ahora.
 - **Doing:** tareas en las que estamos trabajando.
 - **Review/Testing:** tareas que hay que revisar y probar.
 - **Done:** tareas finalizadas.
- **Tarjetas:** Cada una representa una tarea individual.
- **Columnas:** Representan las diferentes fases del trabajo, habitualmente divididas en "Por hacer" (To Do), "En proceso" (Doing) y "Hecho" (Done).

Límite WIP (Work In Progress): Es la norma más importante de Kanban y consiste en limitar la cantidad de tareas En proceso para evitar hacer demasiadas cosas a la vez.

Beneficios: Limitar el trabajo en curso reduce la sensación de agobio, aumenta el control y mejora la calidad del trabajo.

XP – XTREME PROGRAMMING – CALIDAD DEL CÓDIGO

Es un enfoque que se centra estrictamente en **cómo** se programa.

Prácticas principales:

- **Programación en pareja:** uno escribe (Driver) y el otro revisa y corrige (Navigator).

- **TDD (Test-Driven Development):** Consiste en escribir los tests antes de escribir la función real.
- **Refactorización:** Permite mejorar y limpiar el código.
- **Integración continua:** Varias veces al día.

LEAN: EFICIENCIA Y ELIMINACIÓN DE DESPERDICIOS

Metodología adaptada de las fábricas de Toyota cuyo objetivo es quitar todo lo que no aporta valor para mejorar el flujo de trabajo, aplicada para que el equipo centre en programar, probar y entregar valor al usuario.

Eliminación del Desperdicio (Waste): Se busca suprimir cualquier actividad que consuma tiempo sin aportar valor final, como por ejemplo:

- Trabajo innecesario.
- Reuniones inútiles.
- Excesos de documentación.
- Tiempos de espera o retrabajo derivado de errores.

RESUMEN

- **Scrum:** Organizar el trabajo mediante Sprints.
 - **Kanban:** Flujo de trabajo continuo y visualización a través de un tablero con tarjetas móviles.
 - **XP:** Programar "bien" usando la programación en pareja, los tests y la refactorización.
 - **Lean:** Busca la máxima eficiencia.
-

Patrones de Diseño

Un patrón de diseño es una solución general, reutilizable y probada para problemas recurrentes en el diseño de software.

Es una forma estructurada y ordenada de resolver un tipo de problema habitual.

ELEMENTOS COMUNES DE UN PATRÓN

- **Nombre:** Identifica la solución, como Singleton, Factory o Adapter.
- **Problema:** Explica qué resuelve y en qué situaciones aparece.
- **Idea clave:** Describe cómo el patrón soluciona el problema.
- **Participantes:** Enumera las clases y objetos que intervienen.
- **Consecuencias:** Detalla las ventajas y los posibles inconvenientes de su uso.

BENEFICIOS

- **Desacoplamiento:** Fomenta que el código dependa de abstracciones o interfaces en lugar de detalles concretos.
- **Reutilización:** Un mismo patrón es aplicable en aplicaciones pequeñas, webs o videojuegos.
- **Mantenibilidad:** Ayuda a que otros programadores comprendan rápidamente el diseño del sistema.

PATRONES CREACIONALES

Solucionan el problema de tener un código muy rígido y difícil de testear provocado por el uso excesivo de new. **(Tratan sobre cómo usar new de forma inteligente para no acoplar el código).**

Se centran en cómo se crean los objetos, separando la decisión de "cómo se crea" el objeto del código que lo va a utilizar.

Tipos

- **Singleton:** Una única instancia de un recurso mediante un método estático para acceder a ella. (uso: configuraciones)

- **Factory Method:** Delega la decisión de crear un objeto a un método protegido o abstracto.
- **Abstract Factory:** Sirve para crear familias-grupos de objetos que tienen que ir a juego entre sí (es como una fábrica de fábricas).
- **Builder:** Evitar constructores con exceso de parámetros separando la construcción de un objeto de su representación final.

Ejemplos

- **Singleton:** Crear una conexión única a un puerto para enlazar el programa con el motor de BD.
- **Factory Method:** Para vender distintos tipos de entradas (Estándar, VIP, Backstage). En lugar de esparcir `new EntradaVIP()` por todo el código, se delega esa tarea a una clase "Fábrica".
- **Abstract Factory:** Es como el Factory Method, pero para familias de objetos. Para montar un evento se necesita generar la Entrada, la Pulsera y el Póster. Con este patrón se asegura que si alguien pide el pack de Rock, todos los elementos creados combinen entre sí.
 - **1.La interfaz común (Abstract Factory):** Defines qué debe saber hacer cualquier fábrica de tu sistema.

```
public interface FabricaEvento {

    Entrada crearEntrada();

    Pulsera crearPulsera();

}
```

- **2. Las fábricas concretas:** Creas una fábrica para cada "temática" o "familia".


```
public class FabricaRock implements FabricaEvento {
    public Entrada crearEntrada() { return new EntradaRock(); }
    public Pulsera crearPulsera() { return new PulseraRock(); }
}
```
- **Builder:** hacer un `new Evento("Evento", fecha, null, null, 500, true, null)` es un caos de leer. Con un Builder lo construyes paso a paso: `new EventoBuilder().setNombre("Evento").setAforo(500).build()`.

PATRONES ESTRUCTURALES

Se enfocan en cómo organizar y conectar las clases y objetos para que el sistema pueda crecer sin romperse.

Tipos

- **Adapter:** Soluciona la colaboración entre clases e interfaces creando un adaptador-método que implementa el uso de otro método de una librería.
- **Composite:** Permite que objetos individuales y conjuntos de ellos sean tratados de la misma forma.
- **Decorator:** Permite añadir capacidades nuevas a un objeto de forma dinámica, manteniendo una interfaz común y evitando una jerarquía gigante de subclases.

En otras palabras: cambia la herencia (extends) por la **composición** (meter un objeto dentro de otro).

(La magia ocurre porque **el envoltorio es del mismo tipo que el objeto que envuelve**).

Ejemplos

- **Adapter:** Al aplicar el patrón DAO en un proyecto, en el fondo estás creando una capa de adaptación estructural.
 - **Composite:** Este es el patrón perfecto para la **generación dinámica de menús relacionales**. (En un menú de navegación web hay ítems simples y submenús, el Composite permite tratar ambos elementos de la misma forma a través de una interfaz común).
 - **Decorator:** Añade funcionalidades "en caliente". Por ejemplo, si a una entrada se le quieren añadir extras, en lugar de crear una jerarquía gigante de subclases en el código se usa Decorator instanciando un objeto básico y añadiéndole dinámicamente los extras creando un **"objeto decorado"**, **"objeto enriquecido"** o **"envoltorio"**.
-

Principios Solid, Cohesión Y Acoplamiento

El Problema Real del Software

El mayor desafío en el mundo real no es escribir código, sino mantenerlo.

Los programas no los hace una sola persona, no duran dos semanas y cambian constantemente. Si un programa está mal organizado el proyecto se volverá frágil y cada cambio afectará a muchas partes e introducirá errores.

COHESIÓN Y ACOPLAMIENTO

- **Cohesión:** Indica lo bien relacionadas que están las responsabilidades dentro de una clase. El objetivo es buscar una **alta cohesión**, donde la clase tenga un objetivo claro y no mezcle responsabilidades.
- **Acoplamiento:** Indica cuánto afecta un cambio en una clase al resto del sistema. El objetivo es tener un **bajo acoplamiento**, donde los cambios estén localizados y no provoquen efectos en cadena.
- **Relación:** Los principios SOLID existen fundamentalmente para aumentar la cohesión y reducir el acoplamiento.

SOLID

Es una forma de organizar el programa para que modificarlo en el futuro no sea una pesadilla, y dice cómo repartir adecuadamente las responsabilidades entre las clases.

Aplicar SOLID evita cometer errores típicos como crear clases gigantes, abusar de condicionales o depender directamente de tecnologías específicas.

Al seguir estos principios, las clases son más pequeñas, los cambios se localizan fácilmente y el sistema se vuelve altamente flexible y escalable.

S - Single Responsibility Principle (Responsabilidad Única)

Cada clase debe tener una única responsabilidad.

- **Problema que resuelve:** Cuando una clase hace demasiadas cosas, cualquier cambio puede romper las demás funciones.
- **Efecto:** Fomenta clases pequeñas y aumenta la cohesión del sistema.

- **Por qué esto es importante:** SRP reduce el impacto de cada cambio.

O - Open/Closed Principle (Abierto / Cerrado)

El software debe estar abierto a extensión, pero cerrado a modificación (no tocar el código estable).

- **Problema que resuelve:** No se trata de no tocar nunca el código, sino de evitar modificar código estable cuando añadimos nuevas funcionalidades.
- **Por qué es clave:** Un sistema que no cumple OCP no escala.

L - Liskov Substitution Principle (Sustitución de Liskov)

Una clase hija debe poder sustituir a su clase padre sin que el funcionamiento del programa se vea alterado.

- **Problema que resuelve:** Evita herencias incorrectas que se hacen sin pensar en el comportamiento.
- **Por qué es importante:** Frameworks y librerías confían en estas promesas. Romperlas genera errores difíciles de detectar.

I - Interface Segregation Principle (Segregación de Interfaces)

Es mejor tener muchas interfaces pequeñas y específicas que tener una sola interfaz gigante y genérica. Ninguna clase debería implementar métodos que no utiliza.

- **Problema que resuelve:** Evita las interfaces enormes que obligan a las clases a implementar "contratos" o funciones que no les corresponden, reduciendo dependencias innecesarias.
- **Por qué importa:** Reduce dependencias (bajo acoplamiento) y hace el sistema más flexible.

D - Dependency Inversion Principle (Inversión de Dependencias)

Los módulos de alto nivel no deben depender de módulos de bajo nivel; ambos deben depender de abstracciones (interfaces).

- **Problema que resuelve:** Evita depender de tecnologías concretas (como MySQL directamente), lo que impediría cambiar de herramienta en el futuro sin reescribir todo el código.
 - **Por qué es clave** Las tecnologías cambian, el negocio no debería romperse por ello.
-

"Code Smells"

Es un síntoma de que el código tiene un problema de diseño o estructura, a pesar de que funcione correctamente.

El Problema Real: Un code smell no rompe el programa hoy, pero provocará que se rompa en el futuro.

Un código con muchos *smells* es difícil de entender, muy caro de modificar, genera errores colaterales y provoca miedo a tocarlo (lo que equivale a un proyecto "muerto").

DEFERENCIAS CLAVE: BUG – CODE SMELL – DEUDA TÉCNICA

- **Bug:** Error visible e inmediato donde el programa falla de forma evidente.
- **Code Smell:** El programa no falla ahora, pero su impacto dificulta gravemente el mantenimiento.
- **Deuda Técnica:** No falla al inicio, pero es la acumulación de "ya lo arreglaremos " que dificulta progresivamente el mantenimiento.

CATÁLOGO COMÚN DE CODE SMELLS

- **Long Method (Métodos largos):** Funciones de más de 30-40 líneas que no son reutilizables ni testeables.
- **God Class (Clases Dios):** Una clase que controla todo (lógica, bases de datos, usuarios). Viola el principio de Responsabilidad Única (SRP) y tiene una cohesión muy baja.
- **Duplicated Code (Código duplicado):** Copiar y pegar lógica obliga a realizar múltiples cambios si cambian las reglas.
- **Large Parameter List:** Funciones que reciben muchísimos parámetros. Se soluciona agrupándolos en un objeto o DTO (*Data Transfer Object* - *Objeto de Transferencia de Datos*).
- **Feature Envy:** Ocurre cuando una clase utiliza más datos de otra clase que los suyos propios.
- **Primitive Obsession:** Abuso de tipos primitivos (como *strings* o enteros) para representar estados (para lo que se recomienda usar Enums).
- **Switch/If complejos:** Anidaciones largas y complejas. Se soluciona aplicando Polimorfismo o el patrón de diseño Strategy.

- **Nombres pobres:** Usar variables sin contexto claro como data, x o tmp.

ANÁLISIS ESTÁTICO DE CÓDIGO

Es un proceso automático que analiza el código fuente sin llegar a ejecutarlo y sirve para detectar bugs potenciales, vulnerabilidades, code smells y malas prácticas de forma temprana.

El análisis estático de código es, básicamente, añadir al programa un corrector ortográfico y gramatical extremadamente avanzado.

Frente al Análisis Dinámico, el estático previene y es más barato al no requerir ejecución.

HERRAMIENTAS

- **SonarQube**

Es una plataforma profesional multilenguaje utilizada por empresas que analiza exhaustivamente bugs, vulnerabilidades, code smells, duplicaciones, complejidad y calcula la deuda técnica.

Establece unas reglas mínimas “**Quality Gate**” que el código debe cumplir para ser aprobado.

- **ESLint**

Es un analizador estático específico para JavaScript/TypeScript que muestra avisos en tiempo real, ideal para prevención inmediata y aprendizaje.

Detecta variables no usadas, errores de alcance y estilos inconsistentes.

FLUJO DE TRABAJO

1. El programador escribe el código.
2. **ESLint** detecta errores menores en tiempo real.
3. El código corregido se sube al repositorio.
4. **SonarQube** realiza un análisis global del repositorio.
5. El **Quality Gate** decide si el proyecto tiene la calidad suficiente para ser aprobado.
6. Si no pasa, el equipo realiza la **Refactorización** necesaria.

Refactorización

Es el proceso de mejorar la estructura y la calidad general del código para hacerlo más mantenible sin modificar su comportamiento.

Lo que NO es Refactorizar

Corregir un bug no es refactorización porque esto cambia el comportamiento del programa.

Añadir funcionalidades tampoco porque modifica el comportamiento.

TÉCNICAS BÁSICAS DE REFACTORIZACIÓN

- **Rename:** Consiste en cambiar el nombre de las variables, métodos o clases para que reflejen de forma clara y precisa su función real.
- **Extract Method:** Se basa en tomar fragmentos de código complejos y extraerlos a métodos independientes y más pequeños para reducir la complejidad general.
- **Extract Variable:** Implica guardar expresiones que son largas o complejas dentro de variables con un nombre descriptivo para facilitar su lectura.
- **Extract Class:** Consiste en separar las responsabilidades creando nuevas clases cuando una sola clase actual está haciendo demasiadas cosas distintas.

USO DE ENTORNOS DE DESARROLLO (IDEs)

Realizar refactorizaciones de forma manual es muy arriesgado, por lo que deben usarse las herramientas que traen los IDEs para garantizar seguridad y coherencia en los cambios.

- **IntelliJ IDEA:** Su atajo principal es Ctrl + Alt + Shift + T. Permite renombrar, extraer métodos o cambiar firmas, actualizando todas las referencias automáticamente y evitando errores.
- **Eclipse:** Utiliza atajos específicos como Alt + Shift + R para Rename, Alt + Shift + M para Extract Method y Alt + Shift + L para Extract Variable.

BUENAS PRÁCTICAS AL REFACTORIZAR

Solo se debe refactorizar cuando el código ya funciona correctamente.

Hacer cambios pequeños y progresivos, apoyándose en las herramientas del IDE.

Nunca mezclar el proceso de refactorización con la creación de nuevas funciones.

Conclusión profesional: Un buen programador hace que el código funcione, pero un programador profesional hace que ese código se pueda mantener en el tiempo.

Pruebas de software: verificación y validación

EL CONCEPTO DE LA CALIDAD Y PRUEBAS

- **La Calidad:** En software, la calidad no significa simplemente la ausencia de errores, sino un "grado de confianza". Un software de calidad es aquel que cumple su objetivo, se comporta como se espera y puede mantenerse o evolucionar sin romperse.
- **El Testing (Pruebas):** Es un proceso sistemático cuyo objetivo es detectar defectos para que no lleguen a convertirse en fallos, reducir el riesgo y aumentar la confianza en el sistema.
- **Enfoque:** Se trata de formular hipótesis respondiendo a qué puede fallar, en qué condiciones y qué impacto tendría.

NIVELES TÉCNICOS DEL PROBLEMA

- **Error:** Es un fallo humano de lógica, de comprensión, o simplemente un despiste al teclear.
- **Defecto (Bug):** Consecuencia del error humano plasmada en el código fuente (puede existir durante meses sin que nadie se dé cuenta).
- **Fallo:** Comportamiento incorrecto en la ejecución del programa.

VERIFICACIÓN (CONTROL INTERNO)

Es un proceso interno, técnico y objetivo que se realiza durante el desarrollo y cuya misión es comprobar que el software está bien implementado (algoritmos correctos, estructuras que funcionan, datos procesan bien y módulos cumplen su contrato técnico). y sigue el diseño previsto.

Se apoya en pruebas unitarias, de integración y revisiones de código.

VALIDACIÓN (CONTROL EXTERNO)

Es un proceso externo, funcional y contextual que asegura que el software resuelve el problema real y cumple las necesidades del usuario.

Involucra a usuarios o clientes, tiene un componente subjetivo y no suele ser automatizable.

RELACIÓN VERIFICACIÓN-VALIDACIÓN

Ambos procesos no son alternativas, sino complementarios: La verificación es una condición necesaria, mientras que la validación es una condición suficiente.

- **Verificación sin validación:** Da como resultado un software inútil (está bien programado, pero no sirve para lo que el usuario necesita).
- **Validación sin verificación:** Da como resultado un software inestable (hace lo que el usuario quiere, pero falla internamente).

NIVELES DE PRUEBA

- **Unitarias (Nivel código):** ¿Funciona esta pieza en concreto?
- **De integración (Nivel módulos):** ¿Encajan bien las diferentes piezas?
- **Pruebas de sistema (Nivel sistema):** ¿Funciona el todo en su conjunto?
- **Pruebas de aceptación (Nivel usuario):** ¿Sirve para quien lo va a usar?

LA CULTURA DE LA CALIDAD

El primer responsable de la calidad del software es el desarrollador. El tester no arregla el software, sino que se encarga de evaluar los riesgos.

La calidad no se añade al final del proyecto, sino que se construye desde el diseño, el código y las decisiones técnicas iniciales.

Pruebas de Caja negra – Caja blanca

El software puede analizarse desde dos perspectivas:

- **Perspectiva Externa (Usuario):** Representa la filosofía de **Caja Negra**.

No sabe ni le importa cómo está programado internamente. Solo interactúa con pantallas, botones y resultados.

- **Perspectiva Interna (Desarrollador):** Representa la filosofía de **Caja Blanca**.

Conoce perfectamente el código fuente, la estructura y la lógica. Entiende de variables, bucles y condiciones.

PRUEBAS DE CAJA NEGRA - *¿El sistema hace lo que el usuario espera que haga?*

Evalúan el sistema sin conocer el código ni su lógica. El sistema se trata como una caja cerrada donde solo importan las entradas proporcionadas y las salidas obtenidas.

Las realizan: QA (Aseguramiento de Calidad), testers funcionales, clientes o usuarios finales.

- **Técnicas de Caja Negra**
 - **Partición de Equivalencia:** Divide los posibles datos de entrada en "grupos" (particiones) válidos e inválidos, de forma que si un valor de un grupo funciona bien (o da el error esperado), todos los demás valores de ese mismo grupo harán exactamente lo mismo. Por tanto, con que elijas **un solo valor al azar de cada grupo**, ya has probado todo el bloque y te ahorras probar miles de opciones.
 - **Análisis de Valores Límite:** Consiste en probar los límites exactos de una restricción (ej. si el mínimo son 8 caracteres, probar con 7, 8 y 9), ya que ahí suelen ocurrir más errores.

Casos de Uso: Se prueban escenarios y flujos reales de usuario (ej. iniciar sesión, añadir producto y comprar).

PRUEBAS DE CAJA BLANCA - ¿El programa está bien construido?

Analizan el código fuente, la estructura lógica y las rutas de ejecución del programa con el objetivo de garantizar que el código, además de funcionar, sea correcto, mantenible, testeable y predecible a largo plazo.

Las realizan los desarrolladores y equipos técnicos o de arquitectura.

- **Técnicas de Caja Blanca**

- **Cobertura de Sentencias:** Asegura que cada línea individual de código se ha ejecutado al menos una vez durante las pruebas.
- **Cobertura de Decisiones:** Asegura que cada condición (como un if o while) ha sido evaluada tanto en su estado true como en false.
- **Cobertura de Caminos:** Consiste en probar todas las rutas lógicas posibles que puede tomar el flujo del programa, asumiendo que a más condicionales, más caminos y mayor riesgo.

Relación con las pruebas unitarias: La caja blanca se materializa en mediante JUnit con tests unitarios y análisis de cobertura de código.

Comparativa

Característica	Caja Negra	Caja Blanca
Visión	Externa	Interna
Acceso al código	No	Sí
Enfoque	Funcional	Estructural
Quién la usa	Usuarios / QA	Desarrolladores
Detecta	Errores visibles	Errores lógicos
Garantiza	Validación	Verificación

Pruebas de integración, regresión y no funcionales

Cuando un software crece se convierte en un conjunto de módulos que interactúan entre sí, y es en la interacción de estos módulos donde suelen aparecer los problemas más importantes del desarrollo.

Para solucionar esto surgen tres tipos de pruebas avanzadas: de integración, de regresión y no funcionales.

PRUEBAS DE INTEGRACIÓN

Comprueban que los diferentes módulos del sistema funcionan correctamente cuando trabajan juntos, evaluando su interacción.

Son necesarias porque muchos errores aparecen al momento en que una clase llama a otra, se intercambian datos o se accede a recursos compartidos.

- **Enfoques principales:**
 - **Ascendente (Bottom-Up):** Se prueban primero los módulos más pequeños y se combinan progresivamente hasta el sistema completo.
 - **Descendente (Top-Down):** Se empieza probando los módulos superiores, validando pronto la lógica general.
- **Integración continua:** En entornos modernos, las pruebas de integración se ejecutan automáticamente cada vez que se modifica el código.

PRUEBAS DE REGRESIÓN Y CI/CD

Se ejecutan después de cada modificación del código para garantizar que las funcionalidades que ya operaban correctamente no se han roto (regresión).

Se basan en pruebas ya existentes, se ejecutan de forma repetitiva en cada nueva versión y deben estar obligatoriamente automatizadas.

- **Relación con CI/CD (Integración y Despliegue Continuo):**

- **CI:** Cada vez que el desarrollador sube código, el sistema compila y ejecuta automáticamente las pruebas.
- **CD:** Si todas las pruebas pasan correctamente, el sistema puede desplegarse automáticamente.

Este proceso convierte las pruebas en un control de calidad automático que impide subir código roto a producción.

PRUEBAS NO FUNCIONALES

No responden a qué hace el sistema, sino a *cómo lo hace*, evaluando atributos globales de calidad en lugar de funciones concretas.

Tipos principales:

- **Rendimiento:** Evalúan tiempo de respuesta, capacidad bajo carga y consumo de recursos. Incluyen test de carga (muchos usuarios), estrés (situaciones extremas) y *soak* (uso prolongado).
- **Seguridad:** Detectan vulnerabilidades como inyección SQL, errores de autenticación o fugas de datos.
- **Usabilidad y Compatibilidad:** Evalúan facilidad de uso, claridad, comportamiento en distintos entornos y experiencia del usuario.

RESUMEN

La **integración** asegura que las piezas encajan, la **regresión** que los cambios no rompan el sistema y las **no funcionales** que el sistema es profesional, seguro y eficiente.

En proyectos reales, la aplicación de estas pruebas marca la diferencia fundamental entre un "software que simplemente funciona" y un "software profesional".

Métricas de calidad

COMPLEJIDAD CICLOMÁTICA (CC)

Mide el número de caminos que puede seguir el flujo de ejecución del código.

A mayor cantidad de caminos, se necesitan más pruebas para cubrirlos todos, lo que se traduce en un mayor riesgo de fallos.

Fórmula Teórica resumida: n° de decisiones + 1

Cuentan como una decisión: Sentencias if, else if, for, while, case, catch y operadores lógicos como && o ||.

MÉTRICA CRAP

Mide el riesgo y el esfuerzo necesario para modificar un trozo de código basándose en la relación entre la complejidad y su porcentaje de cobertura de pruebas (coverage).

Esta métrica nos la da la herramienta de SonarQube.

Niveles de CRAP

- **1 - 10 (Sana):** Código profesional y bien testado.
- **11 - 20 (Moderada):** Requiere revisión.
- **21 - 30 (Alta):** Código difícil de probar.
- **>30 (Crítica):** Necesita refactorización urgente.

HERRAMIENTAS DE MEDICIÓN EN IDEs

Es necesario instalar el plugin de **SonarQube** en el IDE. Una vez instalado, se puede usar la opción "Analyze with SonarQube/Coverage" haciendo clic derecho en el código.

JaCoCo (Java Code Coverage): Es un plugin de Maven (se añade en el pom.xml) que mide la cobertura de código en proyectos Java, indicando qué porcentaje del código está siendo ejecutado por tus tests.

Depuración con IDEs y pruebas automatizadas con Junit

DEPURAR (o DEBUGGING): Consiste en entender cómo fluye el programa y validar hipótesis con el objetivo de identificar, aislar y corregir defectos de código.

DEPURACIÓN EN INTELLIJ IDEA

Breakpoints (Puntos de interrupción): Son puntos (indicados con un punto rojo en el margen izquierdo) donde la ejecución del programa se detiene por completo. Deben usarse estratégicamente: al inicio de bucles, en condiciones críticas, antes de los return o donde cambian variables importantes.

Controles de Flujo:

- **Step Over (F8):** Ejecuta la línea actual y pasa a la siguiente sin entrar a inspeccionar métodos internos.
- **Step Into (F7):** Entra dentro del método que se está llamando para ver su ejecución.
- **Step Out (Shift+F8):** Sale inmediatamente del método actual en el que te encuentras.

Inspección de Variables: El modo *Debug* permite ver el valor actual de una variable, modificarlo en tiempo real y evaluar expresiones. Esto es clave para detectar variables mal inicializadas, fallos de cálculo o excepciones graves como `NullPointerException` antes de que el programa explote.

PRUEBAS AUTOMATIZADAS CON JUNIT

JUnit Es el framework de testing más utilizado para proyectos en Java.

Permite automatizar validaciones de código, integrarse fácilmente con constructores como Maven o Gradle, y ejecutar cientos de tests de forma automática en apenas unos segundos.

Estructura AAA (El estándar profesional): Todo test debe seguir el patrón AAA:

- **Arrange (Preparar):** Preparar los datos y objetos necesarios.
- **Act (Actuar):** Ejecutar la acción o método a probar.
- **Assert (Verificar):** Comprobar que el resultado obtenido es el esperado.

Métodos de Validación (Assertions)

- **assertEquals(esperado, real) / assertEquals(...):** Compara que el resultado obtenido sea (o no sea) exactamente igual al esperado.
- **assertTrue(condicion) / assertFalse(condicion):** Valida que una condición sea verdadera o falsa.
- **assertNull(objeto) / assertNotNull(objeto):** Comprueba si un objeto es nulo o si tiene valor.
- **assertThrows(...):** Se utiliza para comprobar que el código lanza correctamente una excepción concreta ante un error (ej. al dividir entre cero).
- **assertAll():** Permite agrupar varias validaciones juntas en un solo bloque.

Anotaciones de Ciclo de Vida

- **@Test:** Indica que ese método es una prueba a ejecutar.
- **@BeforeEach / @AfterEach:** Se ejecutan automáticamente justo antes y justo después de **cada** prueba individual (ideal para preparar configuraciones como inicializar objetos).
- **@BeforeAll / @AfterAll:** Se ejecutan una única vez al principio o al final de **todos** los tests de la clase (deben ser estáticos).

CICLO DE TRABAJO PROFESIONAL EN EL TESTING

1. Se detecta un fallo o bug en el sistema.
 2. Se aísla el error visualizándolo con Debugging
 3. Se corrige el código afectado.
 4. Se crea una prueba automatizada en JUnit
 5. El código y su test se suben al sistema de Integración Continua (CI/CD).
 6. El sistema se mantiene estable a largo plazo garantizando la calidad.
-

TDD, ATDD y BDD aplicados al desarrollo web ... “N/A”

CAMBIO DE PARADIGMA

El enfoque tradicional: Consistía en programar, probar al final y luego corregir errores. Esto generaba problemas porque los errores se detectaban tarde, su coste de corrección era alto y el código resultaba difícil de mantener.

El enfoque moderno invierte el orden: primero se escribe la prueba, se ve que falla, se escribe el código mínimo para que pase y finalmente se refactoriza.

TDD (TEST-DRIVEN DEVELOPMENT)

Es una metodología donde se escribe la prueba unitaria antes de escribir el código funcional.

El Ciclo de TDD (Red - Green - Refactor):

- **RED (Rojo):** Se escribe una prueba que va a fallar (porque la funcionalidad aún no existe).
- **GREEN (Verde):** Se escribe el código mínimo e indispensable para conseguir que la prueba pase con éxito.
- **REFACTOR (Refactorizar):** Se mejora y limpia el código manteniendo la prueba en verde (asegurando que no se rompe nada).

ATDD (Acceptance Test-Driven Development)

Este enfoque se basa en que las pruebas se definen a nivel funcional directamente junto con el cliente antes de empezar a programar (ej. qué pasa si hay saldo o si no hay saldo en una transferencia).

Herramientas: Cucumber y Robot Framework.

BDD (Behavior-Driven Development)

Utiliza lenguaje natural estructurado para describir el comportamiento del sistema con el objetivo de mejorar la comunicación entre los desarrolladores, QA (Testing) y el cliente.

Estructura GWT:

- **Given (Dado):** Un contexto inicial.
- **When (Cuando):** Ocurre una acción.
- **Then (Entonces):** Se espera un resultado.

FLUJO DE TRABAJO:

1. El Analista escribe los escenarios usando lenguaje **BDD**.
 2. El Cliente los lee y los valida.
 3. El Desarrollador implementa el código utilizando **TDD**.
 4. Las pruebas se ejecutan de forma automática en el sistema **CI/CD**.
 5. Si alguna prueba falla, el sistema impide hacer el *merge* (no se suben los cambios).
-

UML- Introducción y elementos de los diagramas de clases

El Modelado de Software es el proceso de representar un sistema de software mediante diagramas o modelos para comprender su estructura y funcionamiento antes de implementar el código.

Modelar antes de programar permite entender el sistema, comunicar ideas de forma clara dentro del equipo, detectar errores de diseño de manera temprana y organizar el código antes de escribirlo. Además, visualizar la arquitectura evita cometer errores graves durante el desarrollo, como la duplicación de lógica o la creación de dependencias innecesarias.

INTRODUCCIÓN A UML

El "Lenguaje Unificado de Modelado es un estándar creado en los años 90 que se utiliza mundialmente para representar gráficamente los sistemas de software.

Sus objetivos principales son representar sistemas complejos, facilitar el diseño orientado a objetos, servir como idioma común entre desarrolladores y documentar la arquitectura del proyecto.

Tipos de diagramas UML:

- **De Clases:** Representa la estructura del sistema.
- **De Objetos:** Representa instancias concretas.
- **De Casos de Uso:** Muestra la interacción entre el usuario y el sistema.
- **De Secuencia:** Ilustra la comunicación entre los objetos.
- **De Actividades:** Muestra el flujo de los procesos.
- **De Estados:** Describe el comportamiento de los objetos en distintas fases.

DIAGRAMA DE CLASES

Es el diagrama más utilizado de todos para el diseño orientado a objetos y funciona como el "plano estructural" del software.

Estructura visual: Cada clase se representa con un rectángulo dividido horizontalmente en tres partes: nombre de la clase, atributos y operaciones (métodos).

Elementos Fundamentales

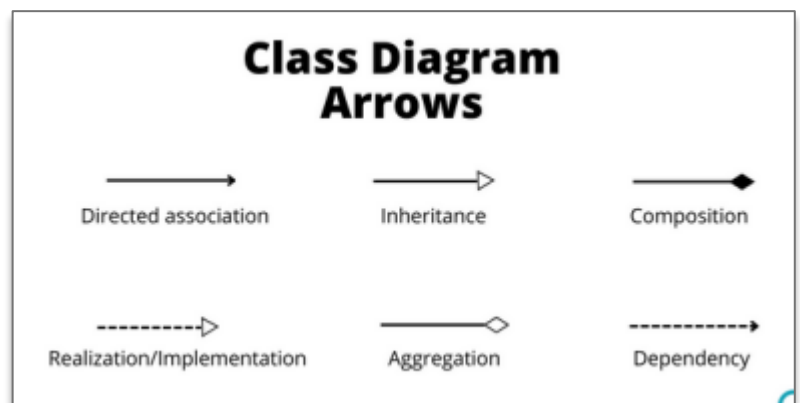
- **Clase:** define propiedades y comportamientos.
- **Objeto:** Instancia concreta creada a partir de una clase.
- **Atributo:** variable.
- **Método:** acciones que puede realizar una clase.

Visibilidad en UML: Determina quién puede acceder a los atributos y métodos:

- **Público:** Se indica con el símbolo +.
- **Privado:** Se indica con el símbolo -.
- **Protegido:** Se indica con el símbolo #.

Relaciones entre Clases: Explican cómo colaboran e interactúan las distintas clases dentro del sistema:

- **Asociación:** Vínculo general.
- **Herencia:** Relación de especialización.
- **Agregación:** Relación "débil" en la que una clase contiene a otra;
- **Composición:** Relación "fuerte" con dependencia total.



Multiplicidad

Indica numéricamente cuántos objetos se relacionan entre sí:

- **1:** Exactamente uno.
- **0..1:** Opcional (cero o uno).
- **1..*:** Uno o muchos.

Diagramas de comportamiento

Los diagramas de comportamiento nos cuentan la historia de cómo funciona el sistema desde distintos ángulos.

Se dividen fundamentalmente en cuatro: Casos de Uso, Secuencia, Actividad y Estado.

DIAGRAMA DE CASOS DE USO (VISIÓN EXTERNA) *¿Qué puede hacer el usuario con el sistema?.*

Es el diagrama más fácil de entender; es como mirar una app desde fuera sin importar cómo está programada internamente.

Elementos Principales:

- **Actor:** Quien usa el sistema.
- **Caso de uso:** La acción que realiza.
- **Sistema:** El límite de la aplicación.

Utilidad: Sirve para ver rápidamente quién participa, qué funcionalidades existen y comprobar si falta algo importante.

DIAGRAMA DE SECUENCIA (INTERACCIÓN INTERNA) *¿Qué mensajes se envían los objetos, quién se comunica con quién, en qué orden temporal?*

Muestra la "conversación" interna del sistema y el comportamiento técnico.

Representa el orden temporal mediante "líneas de vida" (lifelines), mensajes y llamadas a métodos con sus respectivas respuestas.

DIAGRAMA DE SECUENCIA (FLUJO LÓGICO) *¿Qué pasos sigue un proceso y qué decisiones o bifurcaciones aparecen en el camino?*

Se parece mucho a un diagrama de flujo tradicional.

Es ideal cuando un proceso tiene decisiones, errores, caminos alternativos o validaciones.

- **DIAGRAMA DE ESTADOS (CICLO DE VIDA)** *¿Por qué fases o estados pasa un objeto concreto a lo largo de su vida?*

A diferencia del resto, no habla del sistema entero ni de un proceso completo, sino que se centra en la evolución de un único objeto o entidad.

Modela los estados, las transiciones y los eventos que provocan esos cambios.

TABLA COMPARATIVA - RESUMEN

Tipo de Diagrama	Pregunta que Responde	Enfoque / Analogía
Casos de Uso	¿Qué puede hacer el usuario?	Vista Externa: Lo que quiere hacer el usuario.
Secuencia	¿Quién se comunica con quién y en qué orden?	Interacción Interna: Cómo se hablan las piezas/componentes.
Actividad	¿Qué pasos y decisiones tiene el proceso?	Flujo Lógico: El camino paso a paso que sigue el proceso.
Estado	¿Cómo cambia un objeto con el tiempo?	Ciclo de Vida: La evolución y cambios de una única entidad.
